

Code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 int main()
8 {
9     pid_t pid;
10
11     printf("Parent process started.\n");
12     printf("Parent PID : %d\n", getpid());
13     printf("Parent PPID : %d\n\n", getppid());
14
15     pid = fork();
16
17     if (pid < 0)
18     {
19         perror("fork failed");
20         return 1;
21     }
22
23     if (pid == 0)
24     {
25         // Child process
26         printf("----- CHILD PROCESS ----- \n");
27         printf("Child PID : %d\n", getpid());
28         printf("Child PPID : %d\n", getppid());
29
30         printf("Child is running...\n");
31
32         sleep(5);
33
34         printf("Child is running again after waiting.\n");
35
36         sleep(2);
37
38         printf("Child process terminating.\n");
39         exit(0);
40     }
41     else
42     {
43         // Parent process
44         printf("----- PARENT PROCESS ----- \n");
45         printf("Parent PID : %d\n", getpid());
46         printf("Child PID : %d\n", pid);
47
48         printf("Parent is waiting for child...\n");
49
50         wait(NULL);
51
52         printf("Child process terminated.\n");
53         printf("Parent process terminating.\n");
54     }
55
56     return 0;
57 }
```

Output:

```
Parent process started.
Parent PID : 705
Parent PPID : 390

----- PARENT PROCESS -----
Parent PID : 705
Child PID : 706
Parent is waiting for child...
----- CHILD PROCESS -----
Child PID : 706
Child PPID : 705
Child is running...
Child is running again after waiting.
Child process terminating.
Child process terminated.
Parent process terminating.
```